COMPUTER ENGINEERING
QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE-010 Data Structures and Algorithms | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | 9/8/2026 | **DATE SUBMITTED:** | 9/8/2026 |
| **NAME:** | Alvarez, Joshua Caleb P. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. 7.1** **Hands-on Activity 7.1 Sorting Algorithms Pt1**

## 1. PROCEDURE OUTPUT

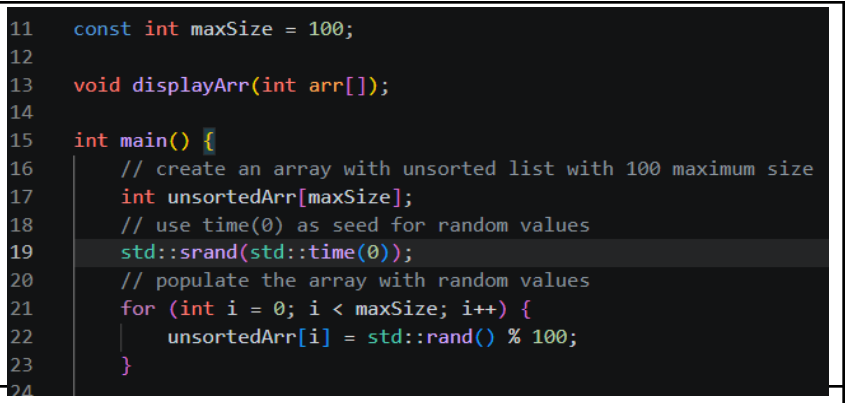
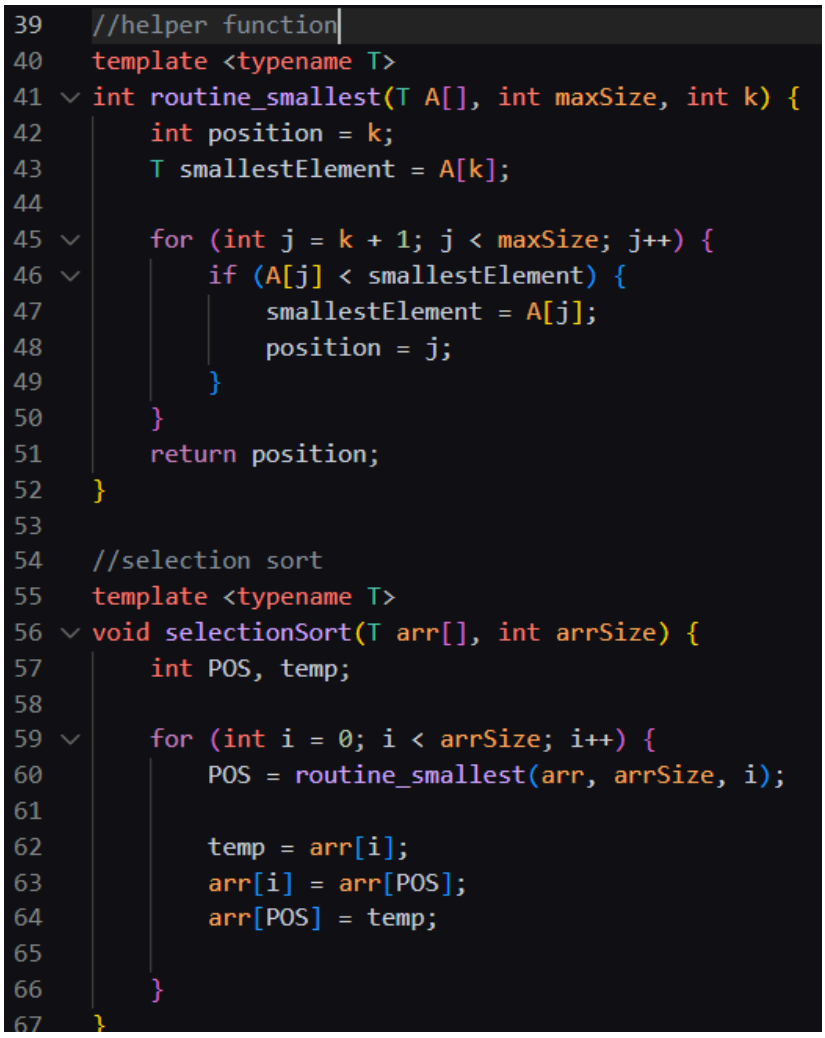
| | |
|---|---|
| **Code and Console Screenshot** |  |
| **Analysis** | The code generates an array of integers where there can be a maximum 100 integers, and then populates it with randomly generated integers from 0 to 99. srand(time(0)) function is used in order to get a different set of random numbers each time the program runs. A for loop is run for all 100 positions in the array, and a random number is stored in each position. |

**Table 7-1. Array of Values for Sort Algorithm Testing**

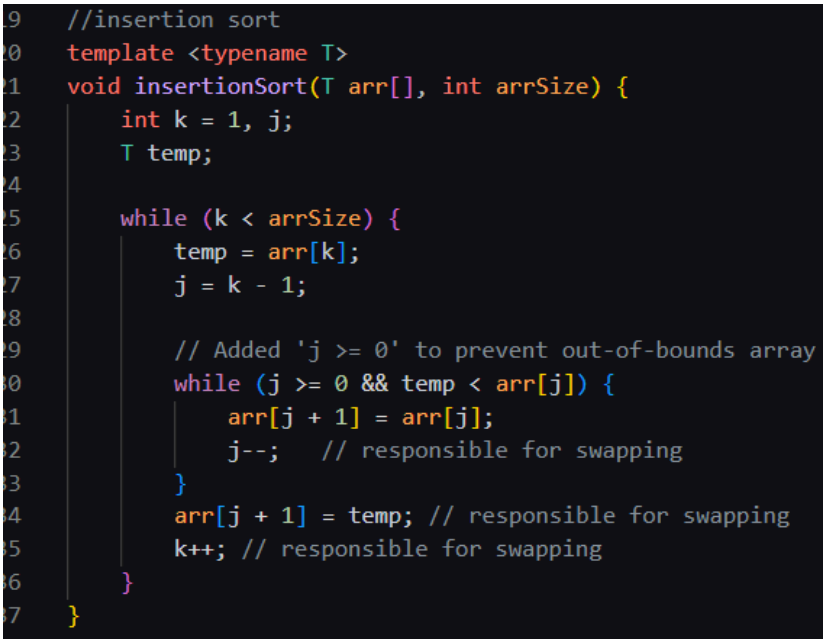
| | |
|---|---|
| **Code and Console Screenshot** |  |
| **Analysis** | **Bubble sort** technique has been used for sorting the elements of an array in increasing order. It repeatedly compares the adjacent pairs and swaps them if the former element is larger than the latter. The number of passes is determined by the outer loop, while comparison and swapping are carried out by the inner loop. However, this technique takes time if the size of the array is large by the time complexity of **$O(n^2)$**. |

**Table 7-2. Bubble Sort Technique**

| | |
|---|---|
| **Code and Console Screenshot** |  |
| **Analysis** | The **selection sort** runs through all 100 items in an outer loop. In every pass, it uses the **routine_smallest** function that searches among the unsorted array elements to find the smallest number. Due to having a nested loop, the comparisons will be approximately equal to n(n-1)/2, resulting in **O(n²)** complexity just as **Bubble sort**. However, since there is one swap operation per every pass, there will be less swapping operations compared to the bubble sort. |

```
39  //helper function
40  template <typename T>
41  int routine_smallest(T A[], int maxSize, int k) {
42      int position = k;
43      T smallestElement = A[k];
44
45      for (int j = k + 1; j < maxSize; j++) {
46          if (A[j] < smallestElement) {
47              smallestElement = A[j];
48              position = j;
49          }
50      }
51      return position;
52  }
53
54  //selection sort
55  template <typename T>
56  void selectionSort(T arr[], int arrSize) {
57      int POS, temp;
58
59      for (int i = 0; i < arrSize; i++) {
60          POS = routine_smallest(arr, arrSize, i);
61
62          temp = arr[i];
63          arr[i] = arr[POS];
64          arr[POS] = temp;
65
66      }
67  }
```

**Table 7-3. Selection Sort Algorithm**

| | |
|---|---|
| **Code and Console Screenshot** |  |
| **Analysis** | **Insertion Sort** works using an outer loop which is executed 99 times, where the value of k starts from 1 since the first element is already sorted. Each time through the outer loop, the inner loop will keep moving all elements which are greater than the temp variable to its right place until it finds the proper location for the temp. The number of executions of the inner loop will vary depending on how many of the arrays are already sorted. The worst case time is n(n-1)/2, which means that it takes **$O(n^2)$** time in the worst case, but in near sorted cases, it may perform much better. |

```
//insertion sort
template <typename T>
void insertionSort(T arr[], int arrSize) {
    int k = 1, j;
    T temp;

    while (k < arrSize) {
        temp = arr[k];
        j = k - 1;

        // Added 'j >= 0' to prevent out-of-bounds array
        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;   // responsible for swapping
        }
        arr[j + 1] = temp; // responsible for swapping
        k++; // responsible for swapping
    }
}
```

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**suppMain.cpp**

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "suppH.h"

const int SIZE = 100;
const int CANDIDATE_COUNT = 5;

// Function declaration
void displayArray(int values[], int length);

int main() {
    int votes[SIZE];

    std::string names[CANDIDATE_COUNT] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    srand(static_cast<unsigned>(time(0)));

    // Generate random votes
    for (int x = 0; x < SIZE; x++) {
        votes[x] = (rand() % CANDIDATE_COUNT) + 1;
    }

    std::cout << "Original Votes:\n";
    displayArray(votes, SIZE);
    // Sort the votes
    insertionSort(votes, SIZE);
    std::cout << "\nVotes After Sorting:\n";
    displayArray(votes, SIZE);
    // Count the votes
    getVoteResults(votes, SIZE, names);

    return 0;
}

// Display the array
void displayArray(int values[], int length) {
    for (int x = 0; x < length; x++) {
        std::cout << values[x] << " ";

        if ((x + 1) % 20 == 0) {
            std::cout << "\n";
        }
    }

    std::cout << "\n";
}
```

## suppH.h

.vscode > Supplementary > C SuppH.h > getVoteResults(int [], int, std::string [])

```cpp
#ifndef SUPP_H
#define SUPP_H
#include <iostream>
#include <string>

// Insertion Sort
template <typename T>
void insertionSort(T data[], int length) {
    for (int current = 1; current < length; current++) {

        T selected = data[current];
        int previous = current - 1;
        while (previous >= 0 && data[previous] > selected) {
            data[previous + 1] = data[previous];
            previous--;
        }
        data[previous + 1] = selected;
    }
}
// Count votes from the sorted array
void getVoteResults(
    int data[],
    int length,
    std::string names[]
) {
    int position = 0;
    int mostVotes = 0;
    int winner = 0;
    std::cout << "\n===== VOTE RESULTS =====\n";

    while (position < length) {

        int voteValue = data[position];
        int total = 1;

        position++;

        while (position < length && data[position] == voteValue) {
            total++;
            position++;
        }
        std::cout << names[voteValue - 1]
                  << " = "
                  << total
                  << " votes\n";
        if (total > mostVotes) {
            mostVotes = total;
            winner = voteValue;
        }
    }
    std::cout << "\nWinner: "
              << names[winner - 1]
              << "\nTotal Votes: "
              << mostVotes
              << "\n";
}

#endif
```

## Pseudocode

```
//
START

DECLARE votes[100]

DECLARE names for 5 candidates

GENERATE 100 random votes
    Each vote is a number from 1 to 5

DISPLAY the original votes

SORT the votes using Insertion Sort

DISPLAY the sorted votes

SET position = 0
SET mostVotes = 0
SET winner = 0

WHILE position is less than 100

    GET the current vote number
    SET total = 1

    MOVE to the next position

    WHILE the next vote is the same as the current vote
        ADD 1 to total
        MOVE to the next position
    END WHILE

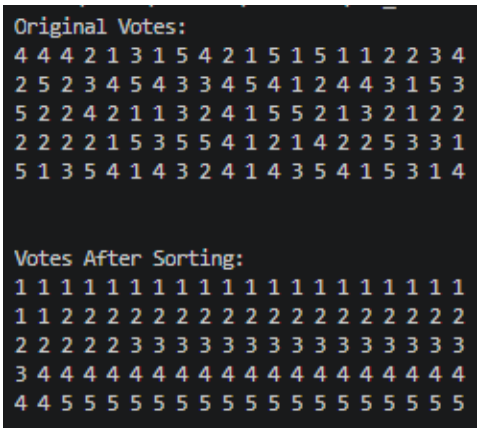
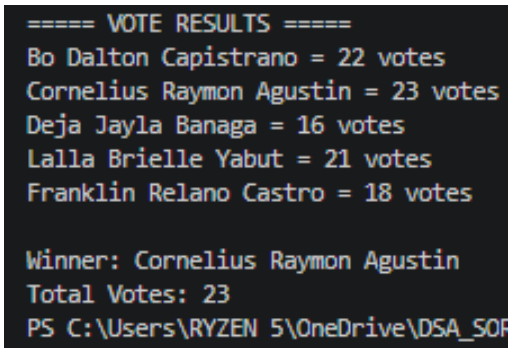
    DISPLAY the candidate name and total votes

    IF total is greater than mostVotes
        SET mostVotes = total
        SET winner = current vote number
    END IF

END WHILE

DISPLAY the winning candidate
DISPLAY the highest number of votes

END
//
```

| Output Console Showing Sorted Array | Manual Count | Count Result of Algorithm |
|---|---|---|
|  | 1 ⟶ 18 votes<br>2 ⟶ 24 votes<br>3 ⟶ 21 votes<br>4 ⟶ 17 votes<br>5 ⟶ 20 votes |  |

**Question: Was your developed vote counting algorithm effective? Why or why not?**

- Yes, my vote counting algorithm was effective because sorting the array first placed the same vote numbers beside each other, which made it easier to count the votes for each nominee. After the Insertion Sort was finished, the program only needed to go through the sorted array once and count the repeated values instead of searching the whole array multiple times. The sorting part has a worst-case time complexity of **O(n²)**, while the vote counting only requires **O(n)** time. Since the array only contains 100 votes and there are only five possible nominees, the algorithm was suitable for the activity. The main limitation is that the votes need to be sorted before using this counting method, because counting the consecutive values would not work properly if the array remained unsorted.

**Conclusion**

Through this activity, I learned how **sorting algorithms** like **bubbleSort**, **selectionSort**, and **insertionSort** can be applied to real-life problems such as vote counting. In particular, Insertion Sort was applied to sort 100 randomly generated votes in ascending order from smallest to largest value. As a result of the process of sorting the values for one candidate appear to be next to each other and it becomes easy to count them. I also learned how to program, compare and to analyze the votes of all candidates and to determine the leader in terms of votes. The activity also helped me understand the importance of the **time complexity** during selection of the sorting algorithm. The worst case complexity of Insertion Sort is **O(n²),** but since the number of elements is not large enough, the algorithm was able to sort them. In addition, comparison between manual counting and counting performed by the program was conducted. Comparison of results was necessary to check if the algorithm was working properly.

COMPUTER ENGINEERING

QUEZON CITY

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Chatgpt, Grammarly**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar correction, code assistance for errors**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Used for debugging, organizing the code, and checking the grammar to make the written content clear and properly structured.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.